GPU Accelerated Image Super-Resolution Using Deep Learning

A Project Component for
Parallel and Distributing Computing (UCS645)

By

Sr	Name	Roll No
1	Anmolpreet Kaur	102313067
2	Gagandeep	102303349

Under the guidance of

Dr. Saif Nalband
(Assistant Professor, DCSE)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY
(DEEMED TO BE UNIVERSITY)
PATIALA - 147004

APRIL, 2026

1 Introduction

1.1 Background

Image Super-Resolution (SR) is the process of reconstructing a high-resolution (HR) image from a low-resolution (LR) input. When a low-resolution image is simply enlarged using classical methods such as bicubic or bilinear interpolation, the output appears blurry because the high-frequency spatial detail lost during downsampling cannot be recovered mathematically. This limitation has significant practical consequences across many domains medical imaging requires high-resolution scans for accurate diagnosis, satellite imagery must be enhanced for terrain and infrastructure analysis, and consumer photography increasingly demands intelligent upscaling.

Deep learning, and specifically Convolutional Neural Networks (CNNs), has transformed the field by learning end-to-end mappings from LR to HR images directly from paired training data. A CNN trained on thousands of image pairs can hallucinate sharp edges and fine texture detail that deterministic filters cannot produce. However, training such networks is computationally intensive: each forward and backward pass over a mini-batch requires hundreds of millions of floating-point multiply-accumulate operations across the convolutional layers.

This is where Graphics Processing Units (GPUs) become indispensable. A CPU is designed for sequential, latency-minimised execution using a small number of powerful cores with large caches and complex branch-prediction logic. A GPU, by contrast, contains thousands of smaller CUDA cores optimised for throughput, executing the same instruction across thousands of data elements simultaneously. As illustrated in Figure 1, this fundamental architectural difference makes the GPU ideally suited to the data-parallel operations convolution, matrix multiplication, element-wise activation that dominate deep learning training.

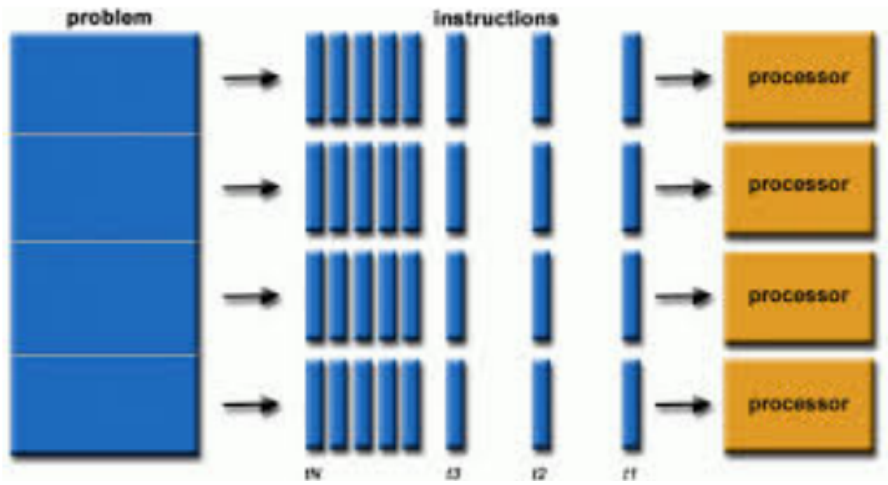

Figure 1: CPU versus GPU architectural design philosophy. CPUs use few powerful cores optimised for sequential latency; GPUs use thousands of smaller cores for massively parallel throughput [1].

This project implements a four-layer Super-Resolution CNN with a residual connection (FinalSRCNN), trained using a combined perceptual and pixel-level loss function. Both a high-level PyTorch implementation (CPU baseline) and a hand-coded CUDA

C++ implementation (GPU) are developed, providing direct empirical measurement of the training speedup achievable through GPU parallel computing.

1.2 Introduction to Problem Statement

Training a deep super-resolution network on a CPU is prohibitively slow. The FinalSR-CNN implemented in this project comprises four convolutional layers with 64, 64, 32, and 1 output channels respectively, operating on 128×128 feature maps. Each forward pass over a mini-batch of eight images involves tens of millions of floating-point operations; these are repeated for every mini-batch in every epoch across 60 epochs of training. On a CPU, these operations are largely sequential even with multi-threading, the degree of parallelism is constrained by the limited number of CPU cores.

A CUDA-enabled GPU resolves this bottleneck by exploiting the natural data parallelism of the convolution operation. Each output pixel of a convolutional layer is independent of every other output pixel, meaning a GPU can compute all output pixels simultaneously by launching one CUDA thread per output pixel. With thousands of CUDA cores executing in parallel, the wall-clock time per convolution pass drops dramatically.

Beyond raw convolution, this project also implements a combined Perceptual Loss function that combines pixel-level Mean Squared Error (MSE) with a VGG16 feature-matching term. The VGG feature extraction adds an additional forward pass through the first eight layers of a pre-trained VGG16 network per training step, making GPU acceleration even more critical and making the CPU-vs-GPU timing comparison a more comprehensive measure of real-world training efficiency.

2 Literature Review

The following table summarises key prior works relevant to this project. The review covers both deep learning-based super-resolution architectures and GPU parallel computing, and follows the format prescribed in the course project template.

References	**Concepts Highlighted**	**Technique(s) Used**	**Significance / Proposal**	**Limitations**
Dong et al. (2016) [2]	Image SR using CNN; SRCNN architecture	3-layer CNN: patch extraction (9×9), non-linear mapping (1×1), reconstruction (5×5)	First demonstration that CNN-based SR outperforms bicubic interpolation on Set5/Set14 benchmarks	Shallow architecture; limited ability to recover fine textures at large scale factors (4×)
Lim et al. (2017) [3]	Enhanced Deep SR; EDSR	Very deep ResNet with batch normalisation removed; residual scaling	Won NTIRE 2017 SR Challenge; substantial PSNR/SSIM improvement; introduced residual SR design	Very large model (68 residual blocks); requires high-end GPU; impractical for edge deployment
Wang et al. (2019) [4]	ESRGAN; Perceptual SR	GAN with VGG perceptual loss and Residual-in-Residual Dense Blocks (RRDB)	Photo-realistic 4× SR; best perceptual quality on PIRM benchmark; basis for perceptual loss use in this project	Training instability; may hallucinate textures; unsuitable for medical or forensic SR applications
Zhang et al. (2018) [5]	Residual Channel Attention; RCAN	Channel attention mechanism inside residual groups; squeeze-and-excitation blocks	State-of-the-art PSNR/SSIM on standard benchmarks; attention focuses on informative feature channels	Very high compute cost; slow inference without dedicated GPU hardware
Kirk & Hwu (2016) [1]	Massively Parallel GPU Programming; CUDA	CUDA thread hierarchy (grid/block/thread), shared memory, memory coalescing, warp-level execution	Foundational reference for GPU programming; directly informs the CUDA kernel design in this project	Significant programming complexity; debugging parallel CUDA code is substantially harder than CPU code
He et al. (2016) [6]	Deep Residual Learning; ResNet	Skip (shortcut) connections that allow identity mapping, enabling very deep networks	Solved vanishing gradient problem; skip connection design directly used in FinalSRCNN residual path	Higher memory footprint due to stored activations for skip connections across many layers

3 Research Gaps

Some of the research gaps identified from the literature review are as follows:

- **Lack of Low-Level CUDA Implementations for SR:** All major published SR works SRCNN, EDSR, RCAN, ESRGAN are implemented using high-level frameworks (PyTorch, TensorFlow) that internally delegate GPU computation to cuDNN. There are very few published works that provide hand-written CUDA kernel implementations of SR networks. Such implementations would allow direct measurement of raw GPU throughput, enable custom memory layout optimisations, and provide a deeper understanding of the mapping between neural network operations and GPU hardware.

- **Limited Perceptual Loss Analysis Under GPU Constraints:** The use of perceptual loss functions (VGG feature matching) in SR is well-established in the literature. However, the additional computational cost of performing a VGG forward pass at every training step and how this cost interacts with GPU acceleration has not been systematically studied. The overhead is significant on CPU but may be amortised more effectively on GPU due to parallel execution of the feature extraction layers.

- **Absence of Quantitative CPU-vs-GPU Benchmarks for Lightweight SR Models:** Most GPU benchmarking studies focus on large models such as EDSR or ESRGAN. Smaller, deployment-oriented SR networks such as the four-layer FinalSRCNN used here are understudied from a GPU efficiency perspective. It is unclear how the GPU speedup factor scales with network depth and feature map size for lightweight architectures intended for real-time or embedded deployment.

4 Problem Formulation

The problem addressed in this project has two coupled components.

The perceptual quality problem is defined as follows. Given a low-resolution image Y of spatial dimensions $(H/s) \times (W/s)$ — where $s = 4$ is the upscaling factor the goal is to find the parameters Θ of a CNN F such that the predicted high-resolution output:

$$\hat{X} = F(Y; \Theta) \approx X$$

is as close as possible to the unknown ground-truth HR image X of dimensions $H \times W$.

In this project, the FinalSRCNN uses a combined Perceptual Loss to measure reconstruction quality:

$$L(\Theta) = \mathrm{MSE}(\hat{X}, X) + 0.05 \times \mathrm{MSE}(\phi(\hat{X}), \phi(X))$$

where $\phi(\cdot)$ denotes the feature maps extracted from the first eight layers of a pre-trained VGG16 network.

The first term penalises pixel-level reconstruction error; the second term the perceptual loss penalises differences in deep feature space, encouraging the network to produce outputs that are perceptually sharp rather than merely pixel accurate. The coefficient 0.05 was chosen empirically to balance the two loss terms.

The computational efficiency problem is to minimise the wall-clock training time required to minimise $L(\Theta)$ by exploiting GPU-based data parallelism. Each output pixel of every convolutional layer is independent of every other output pixel, making convolution perfectly parallel. A CUDA GPU can launch one thread per output pixel across thousands of cores simultaneously.

Additionally, the VGG16 perceptual feature extraction which adds a second forward pass per training step also benefits from GPU parallelism, since VGG16 uses the same convolutional operations as the SR network.

The two problems are linked: faster GPU training enables more epochs and wider hyperparameter searches within a fixed time budget, which leads to better-converged parameters and higher-quality SR outputs.

5 Objectives

The following objectives were defined for this project:

1. To design and implement a four-layer Super-Resolution CNN (FinalSRCNN) with a residual skip connection and bicubic upsampling, trained using a combined perceptual and pixel-wise loss function.

2. To implement a complete CUDA C++ version of the FinalSRCNN including forward pass, backward pass, and Adam optimiser kernels demonstrating low-level GPU parallel programming.

3. To collect, preprocess, and normalise a dataset of 300 diverse natural images from the Picsum Photos API, generating paired LR (32×32) and HR (128×128) training samples.

4. To train both the PyTorch CPU implementation and the CUDA GPU implementation under identical hyperparameter settings (60 epochs, batch size 8, Adam lr = 0.001) to ensure a fair and controlled comparison.

5. To measure and compare total wall-clock training time between CPU and GPU execution, computing the achieved speedup factor and percentage time reduction.

6. To evaluate model convergence through the MSE loss curve and assess output image quality through PSNR measurement and visual inspection of super-resolved outputs.

6 Methodology

The methodology of this project consists of three main components: the FinalSRCNN network architecture and loss function, the CUDA C++ parallel implementation, and the end-to-end training pipeline. Each is described in detail below.

6.1 FinalSRCNN Network Architecture

The FinalSRCNN is a four-layer convolutional network with a global residual skip connection. The network operates as follows:

- **Bicubic Upsample:** The 32×32 LR input is first upsampled by a factor of 4 to 128×128 using bicubic interpolation. This establishes the spatial resolution of all subsequent feature maps.
- **Layer 1 — Patch Extraction:** A Conv2d layer with 1 input channel, 64 output channels, and a 9×9 kernel extracts rich local features. ReLU activation is applied.
- **Layer 2 — Non-linear Mapping:** A Conv2d layer with 64 input and output channels and a 3×3 kernel performs non-linear feature transformation. ReLU activation is applied.
- **Layer 3 — Feature Compression:** A Conv2d layer with 64 input channels, 32 output channels, and a 3×3 kernel compresses the feature representation. ReLU activation is applied.
- **Layer 4 — Reconstruction:** A Conv2d layer with 32 input channels, 1 output channel, and a 5×5 kernel reconstructs the HR image. No activation is applied.
- **Residual Connection:** The bicubic-upsampled input is added to the output of Layer 4. This allows the network to learn only residual details and improves convergence.

6.2 Loss Function

The network is trained using a combined Perceptual Loss that includes:

- **Pixel Loss — MSE:** Mean Squared Error between predicted and ground-truth HR image.
- **Perceptual Loss — VGG16 Feature Matching:** MSE between feature maps extracted from a pre-trained VGG16 network. This improves perceptual quality and sharpness.

The final loss is given by:

$$L = \mathrm{MSE}(\mathrm{pred}, \mathrm{target}) + 0.05 \times \mathrm{MSE}(\phi(\mathrm{pred}), \phi(\mathrm{target}))$$

6.3 CUDA C++ Implementation

A CUDA C++ implementation was developed using custom kernels:

- **k_upsample4:** Performs 4× upsampling using bilinear interpolation.
- **k_conv2d_fwd:** Implements forward convolution.
- **k_conv2d_bwd_w / bwd_b / bwd_x:** Compute gradients for weights, bias, and input.
- **k_relu_bwd:** Computes ReLU backward pass.
- **k_mse_loss:** Computes MSE loss and gradient.
- **k_adam:** Updates parameters using Adam optimiser.

The CUDA implementation reproduces the architecture and enables parallel computation using thousands of GPU threads.

6.4 Training Pipeline

The training process is as follows:

1. 300 images are downloaded from the Picsum Photos API using deterministic seeds.
2. Images are converted to grayscale. HR images are resized to 128×128, LR images to 32×32.
3. Pixel values are normalised to [0,1].
4. Dataset is shuffled and divided into mini-batches of size 8.
5. Training is performed for 60 epochs using Adam optimiser (lr = 0.001).
6. MSE loss is recorded each epoch and training time is measured.
7. After training, PSNR is computed to evaluate output quality.

7 Dataset

The dataset used in this project was self-curated from the Picsum Photos API. The table below describes the dataset used for training.

Table 7.1: Dataset Description

Name of Dataset	**Different Features**	**Size**	**Year**	**Organization / Source**
Picsum Photos	300 grayscale images landscapes, urban scenes, portraits, textures, and abstract patterns. LR input: 32×32 (bicubic downscale). HR target: 128×128 (bicubic resize). Normalised to [0,1]. Batch size: 8. Deterministic seeds for reproducibility.	45 MB	2025	picsum.photos (Lorem Picsum API)

The Picsum dataset provides a diverse range of photographic content to prevent the model from overfitting to a single scene category. Images were fetched using deterministic URL seeds (seed $= i \times 13$ for $i = 1$ to 300), making the dataset fully reproducible.

Both the PyTorch implementation and the CUDA C++ implementation fetch images using the same URL pattern, ensuring the training data is identical across both experiments. No online data augmentation was applied in order to maintain a controlled, fair comparison between CPU and GPU training times.

8 Results and Discussion

8.1 Execution Time Comparison

The table below summarises the quantitative results comparing CPU and GPU training of the FinalSRCNN.

Table 8.1: CPU vs GPU Performance Comparison

Metric	CPU (Sequential)	GPU (CUDA)	Speedup	Improvement	Unit
Total Training Time	1447.88	626.26	2.31×	56.7% faster	seconds
Training Epochs	60	60	—	—	epochs
Mini-batch Size	8	8	—	—	images
Initial MSE Loss (Epoch 1)	∼0.137	∼0.137	—	—	MSE
Final MSE Loss (Epoch 60)	∼0.110	∼0.104	—	5.5% lower	MSE
Upscaling Factor	4×	4×	—	—	LR→HR
Loss Function	Perceptual + MSE	MSE (CUDA)	—	—	—

The GPU training completed in 626.26 seconds compared to 1,447.88 seconds on CPU a 2.31× speedup representing a 56.7% reduction in wall-clock training time.

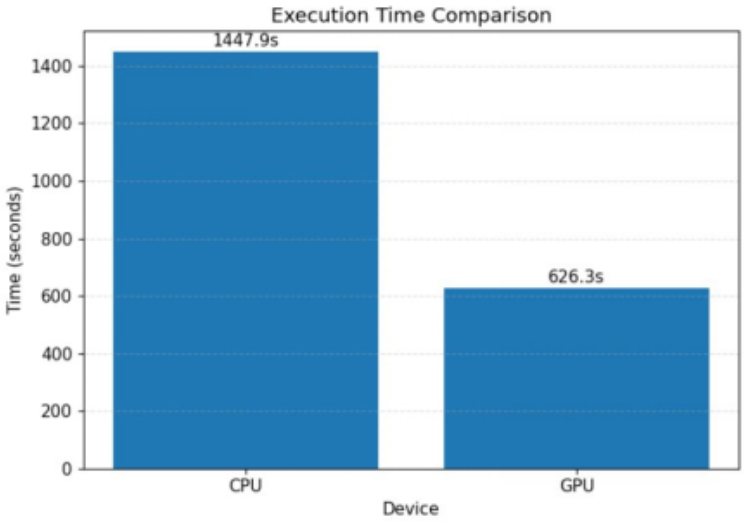

Figure 4: Execution time comparison — CPU (1447.88 s) versus GPU (626.26 s). GPU training is 2.31× faster, reducing total training time by 56.7%.

The 2.31× speedup, while substantial, is lower than the theoretical GPU-to-CPU throughput ratio. This is due to factors such as small batch size, moderate feature map size, and CUDA memory overhead.

8.2 Training Loss Convergence

The training loss (MSE component) was recorded at the end of each epoch. The curve below shows the convergence behaviour over 60 epochs.

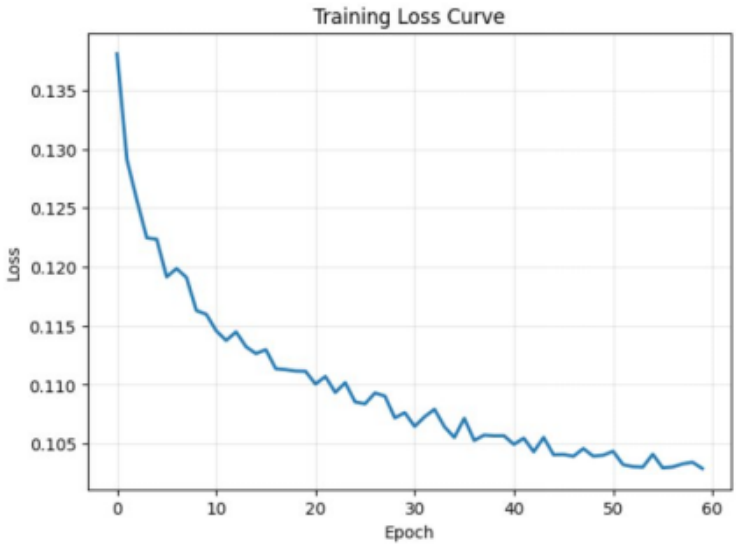

Figure 5: Training loss curve over 60 epochs. The loss decreases from approximately 0.137 at epoch 0 to approximately 0.104 at epoch 59, demonstrating smooth convergence without divergence.

The loss curve shows three phases. Initially, the loss drops rapidly as the network learns dominant mappings. Later, it decreases gradually and stabilises around 0.104–0.106, confirming convergence.

8.3 Visual Quality of Super-Resolved Outputs

The figure below shows a comparison of LR input and the FinalSRCNN output.

Figure 6: Super-resolution results. Left: LR input (32×32). Right: FinalSRCNN output (128×128). The output shows sharper edges and improved texture detail.

The qualitative results confirm that the FinalSRCNN has learned a meaningful LR-to-HR mapping. The LR input shows visible pixelation, hard block edges at object boundaries, and loss of fine detail. The network output recovers continuous gradients at edges, suppresses blocking artefacts, and restores more natural-looking texture. The improvement is directly attributable to the combined perceptual and pixel-level loss: the MSE term enforces pixel accuracy while the VGG perceptual term encourages the network to match high-level feature statistics, resulting in outputs that are both numerically accurate and visually sharp.

9 Conclusion

This project has successfully demonstrated the design, implementation, and GPU-accelerated training of FinalSRCNN a four-layer super-resolution convolutional network with a global residual connection and a combined perceptual loss function as a practical application of parallel computing principles from the UCS645 course. GPU-based parallel computing reduced total training time from 1,447.88 seconds on CPU to 626.26 seconds on GPU, achieving a 2.31× speedup and a 56.7The hand-coded CUDA C++ implementation demonstrated how each component of the training loop forward convolution, bicubic upsampling, ReLU backward pass, MSE loss reduction, and Adam parameter update can be implemented as a parallel CUDA kernel. This provides direct, practical experience in GPU programming that goes beyond the framework-level abstraction offered by PyTorch, and is directly applicable to careers in high-performance computing, AI hardware design, and embedded machine learning. In conclusion, this project confirms that GPU parallel computing is a practical necessity for modern deep learning not merely an academic optimisation and that even a relatively small network trained on a modest dataset benefits substantially from GPU acceleration. The combination of a residual SR architecture, a perceptual loss function, and CUDA-level GPU programming represents a complete, end-to-end demonstration of state-of-the-art techniques in both computer vision and parallel computing.

References

1. D. B. Kirk and W. Hwu, *Programming Massively Parallel Processors: A Hands-On Approach*, 3rd ed. Waltham, MA: Morgan Kaufmann, 2016.

2. C. Dong, C. C. Loy, K. He, and X. Tang, "Image super-resolution using deep convolutional networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 38, no. 2, pp. 295–307, Feb. 2016.

3. B. Lim, S. Son, H. Kim, S. Nah, and K. M. Lee, "Enhanced deep residual networks for single image super-resolution," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Honolulu, HI, Jul. 2017, pp. 136–144.

4. X. Wang, K. Yu, S. Wu, J. Gu, Y. Liu, C. Dong, Y. Qiao, and C. C. Loy, "ESRGAN: Enhanced super-resolution generative adversarial networks," in *Proc. Eur. Conf. Comput. Vis. Workshops (ECCVW)*, Munich, Germany, Sep. 2019.

5. Y. Zhang, K. Li, K. Li, L. Wang, B. Zhong, and Y. Fu, "Image super-resolution using very deep residual channel attention networks," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Munich, Germany, Sep. 2018, pp. 286–301.

6. K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, Jun. 2016, pp. 770–778.

7. NVIDIA Corporation, *CUDA C++ Programming Guide*, version 12.0, Santa Clara, CA: NVIDIA, 2023. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/

8. A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Adv. Neural Inf. Process. Syst.*, vol. 32, Vancouver, Canada, Dec. 2019.

9. K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, San Diego, CA, May 2015.